UAV Software Engineering: ROS 2 Architecture

Advanced Pub/Sub, IPC, and Real-Time Determinism

1 The `rclcpp::Node` Abstraction and DDS Mapping

In ROS 2, every functional component inherits from `rclcpp::Node`. Under the hood, when you call `create_publisher`, the `rclcpp` C++ library translates this request to the `rmw` (ROS Middleware) interface, which maps it to a DDS **DataWriter**. Conversely, `create_subscription` maps to a DDS **DataReader**.

2 Publishers, Topics, and Message Types

A Publisher is an entity responsible for broadcasting data to the ROS 2 network. Instead of sending data to a specific IP address, it publishes to a named channel called a **Topic** (e.g., `/uav/sensors/gps`). The data sent over a topic must adhere to a strict structural definition called a Message Type. While standard ROS 2 provides simple types like `std_msgs::msg::String`, UAVs heavily rely on `sensor_msgs::msg::NavSatFix` for transmitting GPS coordinates.

3 Zero-Copy and Intra-Process Communication (IPC)

A naive implementation publishes messages by value (`publish(msg)`), forcing the DDS middleware to serialize the object into a byte stream (CDR) and send it over the loopback network. For high-bandwidth sensors (4K Cameras, LiDAR) residing in the same physical process, ROS 2 provides **Intra-Process Communication (IPC)**. By instantiating a message as a `std::unique_ptr` and publishing it via `std::move()`, the middleware bypasses the network stack and transfers pointer ownership. This results in **Zero-Copy** message passing, eliminating CPU overhead and maintaining strict real-time latency.

4 The Executor Model and Callback Groups

The **Executor** is the engine that spins the node and orchestrates callbacks. To prevent slow callbacks (e.g., logging) from starving high-frequency control loops, ROS 2 uses **Callback Groups**:

1. **Mutually Exclusive (`MutuallyExclusive`):** Callbacks cannot run in parallel. Crucial for protecting shared memory without aggressive `std::mutex` locking.
2. **Reentrant (`Reentrant`):** The executor can spawn multiple threads to execute the exact same callback concurrently if data arrives at high rates.

5 Wall Timers and Callback Binding

Writing a blocking `while(true)` loop inside a node permanently blocks the Executor's thread. Background tasks must strictly use **Wall Timers** (`rclcpp::TimerBase`), allowing the Executor to fire the logic deterministically.

When registering a timer callback within an Object-Oriented architecture, the Executor needs to know exactly which object instance the method belongs to. You cannot simply pass a method name. There are two ways to bind a member function to the timer:

- `std::bind` **(Legacy):** `std::bind(&MyNode::callback, this)` creates a callable wrapper by explicitly binding the member function pointer to the current object instance (`this`). While common in older ROS 2 tutorials, it is less efficient.

- **Lambda Expressions (Modern C++):** The industry standard is to use an anonymous lambda function: `[this]() { this->callback(); }`. The lambda must capture `[this]` in its capture clause. Without it, the lambda is isolated and cannot access the node's private members (like publishers or state variables). Capturing `[this]` grants the lambda full access to the object's scope, offering cleaner syntax and superior compiler optimization.

6 Real-Time Memory: Custom Allocators (Advanced)

In standard C++, instantiating a message with `new` or `make_unique` uses the OS default allocator. This allocator is non-deterministic: finding a free block of memory in a fragmented heap takes a variable amount of time. In a 1000Hz UAV control loop, a sudden delay of 2ms due to a Page Fault causes a crash. ROS 2 is designed to support **Custom Allocators** (like TLSF - Two-Level Segregated Fit). A true real-time publisher pre-allocates a fixed pool of memory at startup. When publishing, it uses this custom allocator (passed via `rclcpp::PublisherOptions`) to guarantee $O(1)$ constant-time memory allocation, entirely bypassing the OS kernel during flight.

7 Bypassing the Executor: WaitSets (Advanced)

The ROS 2 Executor model carries intrinsic overhead due to mutexes, futures, and thread scheduling. For ultra-low latency flight controllers, relying on the Executor is often unacceptable. ROS 2 exposes the lower-level `rclcpp::WaitSet` API. Instead of letting the Executor invoke callbacks asynchronously, a WaitSet allows the main thread to synchronously block and wait for a specific set of Subscriptions or Timers to trigger. This completely eliminates context-switching and Executor overhead, providing bare-metal deterministic polling—the gold standard for hardware-in-the-loop (HITL) aviation software.

8 ROS 2 Components: Moving from Monolithic Executables to Shared Libraries

In a professional production environment, compiling every node into a standalone monolithic executable is an anti-pattern. If multiple nodes reside in separate binaries, modifying one requires rebuilding and replacing multiple independent binaries, and it introduces architectural friction.

Industry-grade ROS 2 systems leverage **ROS 2 Components**. Instead of writing a standard `main()` function, a node is written as a self-contained C++ class inheriting from `rclcpp::Node` and compiled as a shared library (`.so` file on Linux).

8.1 Key Code Modifications for Components

To transform a traditional node into a loadable component, three core code changes are required:

1. **Header Inclusion:** Include the component registration macro header:

   ```
   #include "rclcpp_components/register_node_macro.hpp"
   ```

2. **Constructor Signature:** The constructor must accept `const rclcpp::NodeOptions & options` and forward it to the base class constructor. This is mandatory so the container can inject runtime configurations (such as enabling Intra-Process Communication):

   ```
   explicit GpsPublisher(const rclcpp::NodeOptions & options)
       : Node("gps_sensor_node", options) { ... }
   ```

3. **Component Registration Macro:** Remove the `int main()` function entirely. At the bottom of the translation unit, register the class as a plugin using the framework's macro:

   ```
   RCLCPP_COMPONENTS_REGISTER_NODE(GpsPublisher)
   ```

9 The Component Container and Dynamic Deployment

Once compiled as a shared library (`libgps_component.so`), the component is loaded at runtime into a **Component Container** (a lightweight management process).

The container instantiates a `ComponentManager` which uses the operating system's dynamic loader (`dlopen`) to open the shared library and instantiate the node inside its own memory space. By loading multiple components (e.g., the GPS sensor and the Flight Controller) into the exact same container process, ROS 2 enables true Zero-Copy IPC: messages passed via `std::unique_ptr` bypass the network stack entirely, achieving $O(1)$ pointer transfer latency.

10 Deployment Cheatsheet: Build and Execution

To build and deploy component-based ROS 2 packages using `colcon` and the command-line interface, follow this operational workflow:

1. **Clean build workspace:**

   ```
   rm -rf build/ install/ log/
   colcon build --packages-select uav_gps
   ```

2. **Source the workspace overlay:**

   ```
   source install/setup.bash
   ```

3. **Launch the Component Container (Terminal 1):**

   ```
   ros2 run rclcpp_components component_container
   ```

4. **Dynamically load the component into the container (Terminal 2):**

   ```
   source install/setup.bash
   ros2 component load /ComponentManager uav_gps GpsPublisher
   ```

11 System Demonstration and Architecture Verification

To empirically prove that the nodes are running within the same process and leveraging Zero-Copy IPC, a three-terminal orchestration model is used:

- **Terminal 1 (The Shared Container):** Hosts the `component_container` process. This serves as the unified runtime memory space where multiple node plugins are injected dynamically.
- **Terminal 2 (The Component Loader):** Issues the runtime command to inject the `GpsPublisher` shared library into the container via ROS 2 services.
- **Terminal 3 (The Flight Controller Subscriber):** Runs the standard executable node `flight_node` which subscribes to the telemetry topic, verifying message flow across nodes.

Verification can be performed using ROS 2 introspection tools. Executing `ros2 node list` displays both nodes, while `ros2 topic info -v /uav/sensors/gps` confirms the underlying middleware transport efficiency and publisher-subscriber topology.